

JAVA 17 OCP · YDS STUDY SERIES

Unit 09 · Collections and Generics — Teknik Hafıza Notları

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya, [ana çift dilli ders notundaki](#) ayrıntıları hızlı tekrar edilebilir bir karar sistemine dönüştürür. Java davranışları Java 17 esas alınarak yazılmıştır. İngilizce terimler için [ünite sözlüğüne](#), cümle yapılarına yönelik çalışma için [grammar notlarına](#) bakın.

1. Önce problemi dört soruya indir

Bir collection sorusunda implementation adı seçmeden önce şu sırayla düşün:

- 1 **Key/value pair var mı?** Varsa `Map`.
- 2 **Duplicate yasak mı?** Varsa `Set`.
- 3 **Ekleme/çıkarma yönü önemli mi?** Varsa `Queue` veya `Deque`.
- 4 Bunların hiçbirisi değil ve position/index önemliyse `List`.

İhtiyaç	Ana interface	Tipik implementation	Hafıza çağırışımı
Index, insertion order, duplicate	<code>List</code>	<code>ArrayList</code>	“Sıra numaralı liste”
Unique element	<code>Set</code>	<code>HashSet</code>	“Üyelik kartı: bir kişiden bir tane”
Unique ve sorted element	<code>Set</code>	<code>TreeSet</code>	“Sıralı benzersiz raf”
FIFO/LIFO veya iki uç	<code>Queue</code> / <code>Deque</code>	<code>ArrayDeque</code>	“İki kapılı koridor”
Key → value lookup	<code>Map</code>	<code>HashMap</code>	“Sözlük: kelime → anlam”
Key → value ve sorted key	<code>Map</code>	<code>TreeMap</code>	“Alfabetik sözlük”

Implementation seçimi

- `ArrayList`: Index ile read işlemi hızlıdır; sona eklemek genel olarak uygundur. Ortaya ekleme/kaldırma element shift gerektirir.
- `LinkedList`: Hem `List` hem `Deque` implement eder. Index ile random access için iyi seçim değildir.
- `HashSet` / `HashMap`: Equality için `hashCode()` ve `equals()` sözleşmesine dayanır; iteration order garantisi yoktur.
- `TreeSet` / `TreeMap`: Natural order veya verilen `Comparator` ile sorted kalır. Temel add/get/remove işlemleri genel olarak $O(\log n)$ dir.
- `ArrayDeque`: Stack ve queue için modern genel seçimdir; `null` kabul etmez.

Memory tip: “Hash hızlı üyelik, Tree sıralı üyelik; List position, Map association, Deque direction.”

2. Equality ile ordering aynı şey değildir

Üç ayrı “eşitlik” mekanizmasını birbirine karıştırma:

Yapı	Aynılığı belirleyen mekanizma	Kritik sonuç
<code>List.contains()</code> gibi genel işlemler	<code>equals()</code>	Aynı value aranır
<code>HashSet</code> , <code>HashMap</code> key	Önce <code>hashCode()</code> , sonra <code>equals()</code>	İki method birbiriyle tutarlı olmalı
<code>TreeSet</code> , <code>TreeMap</code> key	<code>compareTo()</code> veya <code>compare()</code> sonucu 0	Tree yapısı iki object'i aynı kabul eder

`TreeSet`, `equals()` sonucu `false` olsa bile comparator sonucu 0 olan ikinci object'i duplicate sayabilir.

```
record Person(int id, String name) {}

var byId = Comparator.comparingInt(Person::id);
var people = new TreeSet<>(byId);
people.add(new Person(7, "Ada"));
people.add(new Person(7, "Ece"));

System.out.println(people.size()); // 1
```

Kod başarıyla derlenir. `id` değerleri eşit olduğu için comparator 0 döndürür ve `TreeSet` ikinci kaydı eklemes.

3. Mutable, fixed-size ve immutable ayrımı

Oluşturma biçimi	Element replace	<code>add()</code> / <code>remove()</code>	<code>null</code>
<code>new ArrayList<>()</code>	Evet	Evet	Evet
<code>Arrays.asList(array)</code>	Evet	Hayır	Evet
<code>List.of(...)</code>	Hayır	Hayır	Hayır
<code>List.copyOf(collection)</code>	Hayır	Hayır	Hayır
<code>Set.of(...)</code>	Hayır	Hayır	Hayır
<code>Map.of(...)</code>	Hayır	Hayır	Key/value için hayır

Java API belgelerindeki daha kesin terim **unmodifiable**'dir: Bu factory sonuçlarının collection yapısı değiştirilemez; fakat içinde tutulan element object'leri kendi başlarına mutable olabilir. OCP kaynaklarında bu sonuçlar sıklıkla “immutable collection” diye adlandırılır.

`Arrays.asList(array)` özgün array tarafından backed edilen fixed-size bir view'dur: `set()` veya array assignment ile yapılan element değişikliği iki tarafta da görülür.

Buradaki “hayır”ların sonucu aynı değildir:

- Yasak mutation genellikle `UnsupportedOperationException` üretir.
- `List.of(null)` ve benzeri factory çağrıları `NullPointerException` üretir.
- `Set.of("a", "a")` duplicate nedeniyle `IllegalArgumentException` üretir.
- `Map.of(1, "a", 1, "b")` duplicate key nedeniyle `IllegalArgumentException` üretir.

```
var fixed = Arrays.asList("a", "b");
fixed.set(0, "x"); // çalışır
fixed.add("c");    // runtime: UnsupportedOperationException
```

```
var immutable = List.of("a", "b");
immutable.set(0, "x"); // runtime: UnsupportedOperationException
```

OCP trap: Declaration'ın `List` olması mutability garantisi vermez. Gerçek object'in nasıl oluşturulduğuna bak.

4. `remove()` overload tuzağı

`List<E>` iki farklı `remove()` overload'una sahiptir:

```
E remove(int index)
boolean remove(Object value)
```

```
var numbers = new ArrayList<>(List.of(10, 20, 30));
numbers.remove(1);           // index 1: 20 kaldırılır
numbers.remove(Integer.valueOf(1)); // value 1 aranır, bulunamaz
System.out.println(numbers); // [10, 30]
```

Primitive `int`, boxing yapılmadan önce exact `remove(int)` overload'una uyar. Value kaldırmak için `Integer.valueOf(...)` gibi açık bir object kullan.

Bir `LinkedList<Integer>` reference'ı `Queue<Integer>` olarak tutulursa `Queue` interface'inde index overload'u görünmez:

```
Queue<Integer> q = new LinkedList<>(List.of(10, 12));
q.remove(1);           // Integer.valueOf(1) aranır
System.out.println(q); // [10, 12]
```

5. Queue ve Deque: exception mı, özel değer mi?

Queue method çiftleri

İşlem	Exception fırlatan sürüm	Özel değer döndüren sürüm
Ekle	<code>add(e)</code>	<code>offer(e)</code> → başarısızsa <code>false</code>
Başını kaldır	<code>remove()</code>	<code>poll()</code> → boşsa <code>null</code>
Başına bak	<code>element()</code>	<code>peek()</code> → boşsa <code>null</code>

Deque iki uçlu method ailesi

İşlem	Front	Back
Ekle, exception sürümü	<code>addFirst(e)</code>	<code>addLast(e)</code>
Ekle, özel değer sürümü	<code>offerFirst(e)</code>	<code>offerLast(e)</code>
Kaldır, exception sürümü	<code>removeFirst()</code>	<code>removeLast()</code>
Kaldır, <code>null</code> sürümü	<code>pollFirst()</code>	<code>pollLast()</code>
Bak, exception sürümü	<code>getFirst()</code>	<code>getLast()</code>
Bak, <code>null</code> sürümü	<code>peekFirst()</code>	<code>peekLast()</code>

Stack olarak Deque

Stack fikri	Deque method'u	Eşdeğer uç
Push	<code>push(e)</code>	<code>addFirst(e)</code>
Pop	<code>pop()</code>	<code>removeFirst()</code>
Peek	<code>peek()</code>	<code>peekFirst()</code>

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(4);
System.out.println(stack.pop()); // 4
System.out.println(stack.peek()); // 10
```

OCP trap: `pop()` boş deque üzerinde `NoSuchElementException` fırlatır; `poll()` ise `null` döndürür.

6. Map method'larını üç duruma göre çöz

Bir key için önce state'i belirle:

- 1 Key yok.
- 2 Key var, value `null`.
- 3 Key var, value non-null.

Method	Key yok	Value <code>null</code>	Value non-null
<code>get(key)</code>	<code>null</code>	<code>null</code>	Mevcut value
<code>getOrDefault(key, d)</code>	<code>d</code>	<code>null</code>	Mevcut value
<code>putIfAbsent(key, v)</code>	<code>v</code> ekler	<code>v</code> ile değiştirir	Değiştirmez
<code>merge(key, v, f)</code>	<code>v</code> ekler	<code>v</code> ile değiştirir	<code>f(old, v)</code> çağrılır

`get()` tek başına “key yok” ile “key var ama value null” durumlarını ayıramaz:

```
var map = new HashMap<String, String>();
map.put("x", null);

System.out.println(map.get("x"));           // null
System.out.println(map.get("missing"));    // null
System.out.println(map.containsKey("x"));  // true
```

merge() state machine

```
key absent or mapped to null
├── put supplied value; function is NOT called

key mapped to non-null old value
├── result = function(oldValue, suppliedValue)
│   ├── result non-null → store result
│   └── result null    → remove key
```

`null` için üç farklı yeri ayır:

- Çağırdaki `supplied value null` olamaz; `merge(key, null, f)` `NullPointerException` fırlatır.
- Remapping function object'i `null` olamaz; `merge(key, value, null)` `NullPointerException` fırlatır.
- Function object'i non-null olup **çalıştırıldığında** `null` **döndürebilir**; bu durumda mevcut key map'ten kaldırılır.

Yani “function `null`” ile “function result `null`” aynı şey değildir.

```
var scores = new HashMap<String, Integer>();
scores.put("Ada", 10);
scores.put("Ece", null);

scores.merge("Ada", 3, Integer::sum); // 13
scores.merge("Ece", 3, Integer::sum); // 3; function çağrılmaz
scores.merge("Can", 3, Integer::sum); // 3; function çağrılmaz
```

Memory tip: “İki gerçek value varsa merge function; eski value yok/null ise yeni value doğrudan.”

7. Comparable ve Comparator'ı tek tabloda ayır

Özellik	Comparable<T>	Comparator<T>
Package	<code>java.lang</code>	<code>java.util</code>
Abstract method	<code>int compareTo(T other)</code>	<code>int compare(T a, T b)</code>
Parameter sayısı	1	2
Order nerede tanımlı?	Karşılaştırılan class'ın içinde	Ayrı object/lambda
Temel anlam	Natural order	Alternative/custom order

Sonucun anlamı:

- Negatif: ilk value önce gelir.
- Sıfır: ordering açısından eşit.
- Pozitif: ilk value sonra gelir.

Sonucun mutlaka **-1**, **0** veya **1** olması gerekmez.

```
record Book(String title, int pages) implements Comparable<Book> {
    @Override
    public int compareTo(Book other) {
        return title.compareTo(other.title);
    }
}

Comparator<Book> byPages = Comparator.comparingInt(Book::pages);
```

Subtraction comparator tuzağı

Şu kalıp sınav örneklerinde görünse de production açısından güvenli değildir:

```
(a, b) -> a.id() - b.id()
```

Çıkarma overflow üretip işareti bozabilir. Güvenli biçim:

```
Comparator.comparingInt(Item::id)
// veya
(a, b) -> Integer.compare(a.id(), b.id())
```

Comparator chain ve **reversed()**

```
Comparator<Pet> c =
    Comparator.comparing(Pet::name)
                .thenComparingInt(Pet::age)
                .reversed();
```

Bu **reversed()**, bütün chain'i ters çevirir. Yalnız ikinci alanı ters çevirmek istersen:

```
Comparator<Pet> c =
    Comparator.comparing(Pet::name)
                .thenComparing(
                    Comparator.comparingInt(Pet::age).reversed());
```

8. Sorting ve binary search

Collections.binarySearch(list, target) için precondition (ön koşul), list'in aynı natural order ile önceden sorted olmasıdır. Comparator overload'u kullanılıyorsa sort ve search aynı comparator ile yapılmalıdır.

```
var values = new ArrayList<>(List.of(5, 1, 9, 3));
Collections.sort(values); // [1, 3, 5, 9]
System.out.println(Collections.binarySearch(values, 5)); // 2
```

Value bulunmazsa sonuç:

```
result = -(insertionPoint) - 1
insertionPoint = -result - 1
```

Örneğin `[1, 3, 5, 9]` içine `4`, index `2`ye eklenirdi; search sonucu `-3` olur.

OCP trap: Yanlış sırada binary search yapmak exception garantilemez. Sonuç **undefined** olur; belirli bir index'e güvenilemez.

9. Generic declaration anatomisi

Generic class ve record

```
class Box<T> {
    private T value;
    T get() { return value; }
    void set(T value) { this.value = value; }
}

record Pair<K, V>(K key, V value) {}
```

Generic method

Type parameter list, modifiers'tan sonra ve return type'tan önce gelir:

```
public static <T> T identity(T value) {
    return value;
}
```

Class'ın `T`si ile method'un `<T>`si aynı ada sahip olsa bile ayrı declaration olabilir; method type parameter'ı class type parameter'ını shadow eder.

```
class Printer<T> {
    <T> void print(T value) { // method T, class T'yi shadow eder
        System.out.println(value);
    }
}
```

Multiple bounds

Class bound varsa önce yazılır; sonra interface bound'ları gelir:

```
static <T extends Number & Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
```

`<T extends Comparable<T> & Number>` derlenmez; class bound ilk sırada olmalıdır.

10. Invariance: inheritance generic içine taşınmaz

`Integer` extends `Number` doğrudur; fakat:

```
List<Integer> integers = new ArrayList<>();
List<Number> numbers = integers; // DOES NOT COMPILE
```

Bu yasak olmasaydı `numbers.add(3.14)` ile gerçekte `List<Integer>` olan yapıya `Double` eklenebilirdi.

Benzer şekilde implementation type da interface ile uyumlu olmalıdır:

```
List<String> ok = new ArrayList<>();
List<String> bad = new HashSet<>(); // DOES NOT COMPILE
```

Diamond operator yalnız constructor/sağ tarafta kullanılabilir:

```
List<String> ok = new ArrayList<>();
List<> bad = new ArrayList<String>(); // DOES NOT COMPILE
new ArrayList<?>(); // DOES NOT COMPILE
```

11. Wildcard karar sistemi: PECS

PECS: Producer Extends, Consumer Super.

Declaration	Güvenle read	Güvenle add	Hafıza
<code>List<?></code>	<code>Object</code>	Yalnız <code>null</code>	Type bilinmiyor
<code>List<? extends Number></code>	<code>Number</code>	Yalnız <code>null</code>	List number üretir
<code>List<? super Integer></code>	<code>Object</code>	<code>Integer, null</code>	List integer tüketir

Tablodaki `null`, yalnız **compile-time type** açısından eklenebilir demektir. Underlying collection `null`u veya mutation'ı reddediyorsa çağrı runtime'da exception fırlatabilir. Genel kuralda lower bound, `T` ve `T`nin subtype'larını eklemeye izin verir; burada `Integer` final olduğundan somut subtype yoktur.

```
static double sum(List<? extends Number> values) {
    double total = 0;
    for (Number n : values) total += n.doubleValue();
    return total;
}
```

```
static void addDefaults(List<? super Integer> values) {
    values.add(10);
    values.add(20);
    Object first = values.get(0);
}
```

Neden ? `extend`se add yapılamaz?

```
List<? extends Number> values = new ArrayList<Integer>();
values.add(3.14); // DOES NOT COMPILE
```

Reference gerçekte `List<Integer>` tutabildiğinden `Double` eklemek güvenli değildir. Hatta `Integer` eklemek de güvenli değildir; gerçek list `List<Double>` olabilir.

Wildcard assignment örnekleri

```
List<? extends Number> a = new ArrayList<Integer>(); // compiles
List<? super Integer> b = new ArrayList<Number>();   // compiles
List<? super Integer> c = new ArrayList<Object>();   // compiles
List<?> d = new ArrayList<String>();                 // compiles
```

12. Type erasure ve sınav sonuçları

Type erasure (tür silme), generic type bilgisinin çoğunun bytecode düzeyinde silinmesidir. Unbounded `T` çoğunlukla `Object`te, bounded `T` ise ilk bound'a erase edilir. Compiler gerektiğinde cast ve bridge method ekler.

Aynı erasure nedeniyle overload edilemez

```
void process(List<String> x) {}
void process(List<Integer> x) {} // DOES NOT COMPILE: same erasure
```

İki method da runtime signature bakımından `process(List)` olur.

Yapılamayan işlemler

- `new T()` çağrılmaz.
- `new T[10]` oluşturulamaz.
- `new List<String>[10]` oluşturulamaz.
- `obj` değişkeninin compile-time türü `Object` ise `obj instanceof List<String>` yazılamaz; `String` type argument'ı çalışma zamanında doğrulanamaz.
- Class type parameter'ı static context'te doğrudan kullanılamaz.
- Generic class doğrudan veya dolaylı olarak `Throwable` subclass'ı olamaz.

Yapılabilen reifiable kontrol:

```
if (obj instanceof List<?>) {
    System.out.println("Bir List");
}
```

Java 17 sınırı: her parameterized instanceof yasak değildir

Java 16'dan itibaren, unchecked conversion gerektirmeyen bazı parameterized type kontrolleri geçerlidir. Örneğin type argument zaten `String` olarak biliniyorsa:

```
java.util.List<String> names = new java.util.ArrayList<>();
System.out.println(names instanceof java.util.ArrayList<String>); // true
```

Başarıyla derlenir ve `true` yazdırır. Burada kontrol edilen, nesnenin `ArrayList` olmasıdır; öğelerin her birinin `String` olup olmadığı kontrol edilmez. `Object value = names;` ardından `value instanceof java.util.List<String>` ise **Does not compile**. Kararı yalnız sağdaki türe bakarak verme; soldaki ifadenin compile-time türünü de incele. [Java 17 JLS §15.20.2](#).

Raw type

```
List raw = new ArrayList<String>();
raw.add(42); // compiler warning
String s = (String) raw.get(0); // runtime: ClassCastException
```

Raw type çoğu zaman compile error değil warning üretir; type safety kaybının bedeli runtime'da çıkabilir.

13. Compile-time / runtime hızlı ayrımı

Kod veya durum	Sonuç	Neden
<code>List<> x</code>	Does not compile	Diamond solda kullanılamaz
<code>new ArrayList<?>();</code>	Does not compile	Wildcard ile instance oluşturulamaz
<code>List<Number> x = new ArrayList<Integer>();</code>	Does not compile	Generic invariance
<code>extends</code> wildcard üzerinden non-null üye ekleme	Does not compile	Gerçek subtype bilinmez; <code>null</code> tür açısından istisnadır
<code>List.of(...).add(...)</code>	Runtime exception	Immutable factory result
<code>Arrays.asList(...).add(...)</code>	Runtime exception	Fixed-size view
<code>Set.of("x", "x")</code>	Runtime exception	Duplicate factory input
Empty deque üzerinde <code>pop()</code>	Runtime exception	<code>NoSuchElementException</code>
Empty deque üzerinde <code>poll()</code>	Başarılı, <code>null</code>	Özel değerli method
Incomparable object'leri natural <code>TreeSet</code> ekleme	Runtime exception	<code>ClassCastException</code>
Raw list'ten yanlış cast	Runtime exception	Erased type safety
Unsorted list üzerinde <code>binarySearch()</code>	Derlenir; sonuç undefined	Search precondition bozulur

14. Review Questions 1–20 teknik cevap anahtarı

Bu tablo, [kaynak sorularının](#) Java 17 analizidir. Soru 7'de kaynak Appendix'i ile Java override kuralı arasındaki çelişki ayrıca belirtilmiştir.

Soru	Cevap	Kısa teknik gerekçe
1	A, E	Duplicate ürünler için sade <code>List</code> → <code>ArrayList</code> ; sorted ID ve text value için <code>TreeMap</code> .

Soru	Cevap	Kısa teknik gerekçe
2	C, G	<code>List<?></code> element'ı <code>Object</code> görünür; iki <code>String</code> method çağrısı derlenmez. Kalan <code>List.of()</code> mutation'ı runtime'da <code>UnsupportedOperationException</code> verir.
3	B	öla önce <code>pop()</code> ile çıkar; loop sırasıyla <code>hello</code> ve <code>hi</code> yazdırır.
4	B, F	B lower bound, F reference tarafındaki upper bound nedeniyle uyumludur.
5	B	Method'un <code><T></code> si record'un <code>T</code> sinden ayırır; primitive'ler box edilir; raw use yalnız warning üretir.
6	B, F	B doğrudan beak length'i ters sıralar; F bütün name+beak chain'ini ters çevirir ve verilen data için 7,5,3 üretir.
7	F (Appendix: B, F)	F covariant return ile gerçek override'dır. Appendix B,F der; B parameter'ı daraltıp overload oluşturur, override değil.
8	A	Comparator case-insensitive reverse lexicographic order uygular: <code>Abb aab 123</code> .
9	A, B, D	<code>U extends Exception;</code> <code>FileNotFoundException,</code> <code>Exception,</code> <code>NullPointerException</code> uygundur.
10	A, B, E, F	<code>Collection.forEach</code> tek consumer alır. <code>Map.forEach</code> iki argümanlı <code>BiConsumer</code> ister; <code>keys()/valueSet()</code> yoktur.
11	B, E	B lower bound ve E upper bound assignment'ıdır; ikisi de <code>List<?></code> parameter'ına geçebilir.
12	C	<code>t1.compareTo()</code> ile text'e göre 88,55; <code>t2</code> constructor'a verilen comparator ile number'a göre 55,88.
13	A	Natural-order <code>c2</code> ile <code>2</code> index 0'dadır. İki <code>reverse()</code> birbirini götürür.
14	A, B	<code>Z<Y></code> içindeki <code>Y</code> , aynı adlı class'ı shadow eden type parameter'dır; yalnız class <code>Y</code> kullanmayan A/B geçerlidir.
15	A, C	<code>List/LinkedList</code> görünümünde <code>remove(int)</code> index kaldırır; <code>Queue</code> görünümünde <code>remove(Object)</code> value arar.
16	E	<code>Map</code> interface'inde <code>contains()</code> yoktur; line 7 derlenmez.

Soru	Cevap	Kısa teknik gerekçe
17	A, E	Line 56 <code>Map.Entry * 2</code> nedeniyle derlenmez. O satır kaldırılırsa <code>immutable one.replaceAll()</code> runtime exception verir.
18	B	Doğru syntax <code>public static <T> T identity(T t)</code> dir.
19	F	Key 1 için function <code>10+3=13</code> ; key 3 null olduğu için function çağrılmadan 3 atanır.
20	B, D, F	<code>Comparator</code> , <code>java.util</code> içindedir ve iki parameter'lı <code>compare()</code> bildirir.

Soru 7 doğrulama notu

Şu declaration'a `@Override` eklemek Java 17'de compiler error üretir:

```
class Child extends Alpaca {
    @Override
    public List<String> hairy(ArrayList<String> list) {
        return null;
    }
}
```

Hata kategorisi: **Does not compile** — parameter type override sırasında covariant olamaz; aynı olmalıdır. `@Override` kaldırılırsa method ayrı bir overload olarak derlenir.

15. Özgün OCP tarzı mini quiz

Aşağıdaki sorular kaynak sınav soruları değildir; pekiştirme amacıyla hazırlanmış özgün çalışma sorularıdır.

Soru A — iki cevap seçin

Hangi iki satır derlenir?

```
List<? extends Number> a = new ArrayList<Integer>();
List<? super Integer> b = new ArrayList<Number>();
```

- A. `a.add(1);`
- B. `Number n = a.get(0);`
- C. `b.add(1);`
- D. `Integer i = b.get(0);`

Soru B — sonuç nedir?

```
var map = new HashMap<String, Integer>();  
map.put("x", null);  
map.merge("x", 4, Integer::sum);  
map.merge("x", 3, (a, b) -> null);  
System.out.println(map.containsKey("x"));
```

- A. `true`
- B. `false`
- C. Does not compile
- D. `NullPointerException`

Soru C — sonuç nedir?

```
var values = Arrays.asList(3, 1, 2);
values.sort(Comparator.naturalOrder());
System.out.println(Collections.binarySearch(values, 4));
```

- A. 3
- B. -3
- C. -4
- D. Sonuç undefined

Cevaplar

- **A: B ve C.** `extends` reference'ı `Number` üretir; `super Integer` reference'ı `Integer` tüketir. `super` üzerinden read type'ı `Object` tir.
- **B: B.** İlk `merge`, explicit null value'yu doğrudan 4 yapar. İkinci mapping function `null` döndürdüğü için key kaldırılır.
- **C: C.** 4'ün insertion point'i index 3'tür: $-(3) - 1 == -4$.

16. Son 60 saniyelik tekrar

- `List`: duplicate + index; `Set`: unique; `Queue/Deque`: direction; `Map`: key/value.
- `List.of/copyOf`, `Set.of/copyOf`, `Map.of/copyOf` sonuçları unmodifiable'dır ve null öğe/key/value kabul etmez; mutable öğeleri otomatik immutable yapmazlar.
- `Arrays.asList()` fixed-size; `set()` var, `add/remove` yok.
- `remove(1)` için declared type'a bak: index overload'u görünür mü?
- `Queue`'da `remove/element` exception; `poll/peek` null.
- `merge`: eski non-null ise function; `absent/null` ise yeni value doğrudan.
- `Comparable.compareTo(one)`; `Comparator.compare(two)`.
- Binary search'ten önce aynı order ile sort.
- Generic'ler invariant'tır.
- PECS: producer `extends`, consumer `super`.
- Generic method `<T>` return type'tan önce.
- Erasure: aynı raw signature overload edilemez; `new T()` yok.